Narendra Karamala

narendrak7216@gmail.com | +91-7893902908 | LinkedIn

Experience

Rubrik Inc., Bengaluru *Feb 2024 – Feb 2025 (Intern), Feb 2025 – Present (Full-Time)*
Systems Software Engineer

- Worked on large-scale distributed systems and internal databases for Rubrik's data protection platform.
- Improved system stability and performance by identifying and fixing 14 critical bugs in production within a week.
- Investigated and resolved complex customer-reported issues across networking, storage, and system OS, reducing escalation turnaround time.
- Wrote automation scripts to streamline internal debugging and diagnostics.
- Authored internal Knowledge Base articles to support the engineering team workflows.
- Worked on product feature enhancements and contributed to product performance improvements.

Technologies: Distributed Systems, Databases, Linux, Rubrik OS, Networking.

Education

Bachelor of Technology in Computer Science and Engineering *2020 – 2024*
Jain University, Bengaluru.
CGPA:8.3
Courses: Data Structures and Algorithms | Problem Solving | Database Management System | Operating System | Computer Networks | Object Oriented Programming Systems(OOPS)

Skills

Programming: C++, Java, Bash, Javascript.
Systems: Linux, Networking, Distributed Systems
Databases: MySql, MongoDB, Firebase.
Tools: Git/GitHub.
Problem Solving and Algorithms: Solved 400+ Data Structures and Algorithms (DSA) problems focusing on dynamic programming, graphs, trees, and union-find.

Projects

Distributed Key-Value Store with Replication and Recovery

- **Java, Distributed Systems, Databases**
- Designed and implemented a **persistent distributed key-value store** supporting GET, PUT, and DELETE operations.
- Implemented **write-ahead logging (WAL)** to ensure durability and enable crash recovery by replaying logs on restart.
- Built an **in-memory index** for O(1) read and write operations, backed by append-only log storage.
- Designed **leader–follower replication**, enabling consistent writes and data synchronization across nodes.
- Analyzed consistency and availability tradeoffs, documenting failure scenarios such as partial replication and node crashes.

Distributed Rate Limiter (Systems Project)

- **Java, Spring Boot, Redis, Lua, Distributed Systems**
- Designed and implemented a **distributed rate limiter** to enforce per-user API request limits across horizontally scaled services.
- Implemented a **token bucket algorithm** using **Redis Lua** scripts to ensure **atomic request** handling and correctness under high concurrency.
- Built the service as **stateless**, enabling seamless horizontal scaling and consistent rate enforcement across instances.
- Optimized request handling to **O(1) time complexity**, supporting **10K+ requests/second** with sub-5ms latency.
- Implemented **TTL-based cleanup** to prevent stale state and unbounded memory growth in Redis.
- Analyzed failure scenarios including Redis outages and burst traffic, and documented tradeoffs between **fail-open vs fail-closed** behavior.